МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторные работы
по курсу «Информационный поиск»

Тема: Криптовалюты и блокчейн (англ.)

Выполнил: Юнусов Р. К.
Группа: М80-410Б-22
Преподаватель: Кухтичев Антон Алексеевич

Москва, 2026

Содержание

1 Лабораторная работа №1. Добыча корпуса документов **3**
1.1 Задание . . . 3
1.2 Описание метода . . . 3
1.3 Реализация . . . 3
1.4 Результаты . . . 3
1.5 Выводы . . . 3

2 Лабораторная работа №2. Поисковый робот **4**
2.1 Задание . . . 4
2.2 Описание метода . . . 4
2.3 Реализация . . . 4
2.4 Результаты . . . 5
2.5 Выводы . . . 5

3 Лабораторная работа №3. Токенизация **5**
3.1 Задание . . . 5
3.2 Описание метода . . . 5
3.3 Реализация . . . 5
3.4 Результаты . . . 6
3.5 Выводы . . . 6

4 Лабораторная работа №4. Стемминг **6**
4.1 Задание . . . 6
4.2 Описание метода . . . 6
4.3 Результаты . . . 6
4.4 Выводы . . . 7

5 Лабораторная работа №5. Закон Ципфа **7**
5.1 Задание . . . 7
5.2 Описание метода . . . 7
5.3 Результаты . . . 7
5.4 Выводы . . . 8

6 Лабораторная работа №6. Булев индекс **8**
6.1 Задание . . . 8
6.2 Описание метода . . . 8
6.2.1 Формат индекса . . . 8
6.3 Результаты . . . 8
6.4 Выводы . . . 9

7 Лабораторная работа №7. Булев поиск **9**
7.1 Задание . . . 9
7.2 Описание метода . . . 9
7.3 Реализация . . . 9
7.3.1 CLI . . . 9
7.3.2 Веб-сервис . . . 9
7.4 Результаты . . . 10
7.5 Выводы . . . 10

Заключение **10**

1. Лабораторная работа №1. Добыча корпуса документов

1.1. Задание

Скачать корпус документов единой тематики, изучить его характеристики, выделить текст, найти существующие поисковые системы для данного набора. Источников должно быть не менее двух. Объём --- не менее 30 000 статей.

1.2. Описание метода

Для исследования выбрана тема <<Криптовалюты и блокчейн>> на английском языке. Корпус собран из трёх независимых онлайн-изданий:

- **CoinTelegraph** (cointelegraph.com) --- одно из крупнейших медиа о криптоиндустрии;
- **CryptoSlate** (cryptoslate.com) --- аналитический портал о криптовалютах и блокчейне;
- **Bitcoin Magazine** (bitcoinmagazine.com) --- старейшее издание о Bitcoin.

Документы представляют собой HTML-страницы с текстовым контентом, метаинформацией (заголовок, дата публикации, автор) и мультимедиа. Из каждой страницы извлекается заголовок и основной текст.

Существующие поисковые системы по данным источникам:

- Google с ограничением `site:cointelegraph.com`;
- Встроенный поиск CoinTelegraph и CryptoSlate.

Примеры поисковых запросов к Google: `site:cointelegraph.com bitcoin ETF`, `site:crypto ethereum merge`. Встроенные поисковики иногда не поддерживают булеву логику и не показывают релевантность результатов.

1.3. Реализация

Написан асинхронный краулер на Python с использованием `aiohttp` и `BeautifulSoup`. Обнаружение URL происходит через sitemap каждого источника. Конфигурация задаётся через YAML-файл (`crawler/config.yaml`).

1.4. Результаты

Показатель	Значение
Количество документов	33 290
Количество источников	3
Суммарный объём текста	132,8 МБ
Средний размер документа	4 344 байт
Средний объём текста	4 183 байт
Минимальный объём текста	201 байт
Максимальный объём текста	331 878 байт

1.5. Выводы

Собранный корпус содержит 33 290 документов из трёх источников на тему криптовалют и блокчейна. Объём текста составляет 132,8 МБ, что достаточно для построения полноценного поискового индекса и проверки алгоритмов поиска.

2. Лабораторная работа №2. Поисковый робот

2.1. Задание

Написать поисковый робот, получающий путь до YAML-конфига, сохраняющий документы в базу данных, с поддержкой остановки и продолжения обкачки.

2.2. Описание метода

Поисковый робот реализован на Python с использованием асинхронной библиотеки `aiohttp` для параллельной загрузки страниц. Парсинг HTML выполняется с помощью `BeautifulSoup` (парсер `lxml`).

Обнаружение URL-адресов происходит через XML Sitemap каждого источника. Краулер рекурсивно обходит подсайтмапы (до 80 штук) и собирает все доступные ссылки.

2.3. Реализация

Конфигурация задаётся через файл `crawler/config.yaml`:

```
db:
  host: localhost
  port: 27017
  name: crypto_news
sources:
  - name: cointelegraph
    sitemap: https://cointelegraph.com/sitemap.xml
    domain: cointelegraph.com
    target: 15000
```

Документы сохраняются в MongoDB со следующими полями:

- `url` --- нормализованный URL;
- `raw_html` --- сырой HTML-текст;
- `title` --- заголовок;
- `text` --- извлечённый текст;
- `source` --- название источника;
- `crawl_timestamp` --- дата обкачки (Unix timestamp);
- `content_hash` --- MD5-хеш для определения изменений.

Робот может быть остановлен и возобновлён: при повторном запуске он пропускает уже сохранённые URL. Переобкачка реализуется через сравнение `content_hash`.

2.4. Результаты

Показатель	Значение
CoinTelegraph	14 731 документ
CryptoSlate	11 410 документов
Bitcoin Magazine	7 292 документа
Итого	33 290
Concurrency	80 параллельных соединений
Общее время обкачки	≈35 мин

2.5. Выводы

Асинхронный краулер позволил собрать более 33 тысяч документов за 35 минут благодаря высокому параллелизму (80 одновременных соединений) и использованию сайтмапов для обнаружения URL.

3. Лабораторная работа №3. Токенизация

3.1. Задание

Реализовать разбиение текстов документов на токены, описать правила токенизации, привести статистику.

3.2. Описание метода

Токенизация реализована на C++ (STL разрешён для данного этапа). Правила:

1. Разделение по пробелам и знакам пунктуации;
2. Приведение к нижнему регистру;
3. Минимальная длина токена --- 2 символа;
4. Удаление чисто цифровых токенов;
5. Апострофы внутри слов сохраняются (напр. `don't`).

3.3. Реализация

Файл `core/tokenizer.cpp`. Обрабатывает каждый файл из каталога корпуса: пропускает первые две строки (заголовок, URL), оставшийся текст разбивает на токены по указанным правилам. Результат записывается в файл `tokens.txt` --- по одному токену на строку.

3.4. Результаты

Показатель	Значение
Количество токенов	20 763 046
Уникальных токенов	253 465
Средняя длина токена	5,35 символа
Время токенизации	3,43 с
Скорость	≈38 700 КБ/с

3.5. Выводы

Токенизатор обработал 33 290 файлов за 3,43 секунды со скоростью около 38,7 МБ/с. Средняя длина токена --- 5,35 символа, что характерно для английского языка. Для ускорения можно использовать потоковое чтение или многопоточную обработку.

4. Лабораторная работа №4. Стемминг

4.1. Задание

Добавить стемминг в поисковую систему.

4.2. Описание метода

Для стемминга используется оригинальный алгоритм Портера (1980) для английского языка. Алгоритм состоит из пяти шагов, каждый из которых удаляет или заменяет определённые суффиксы слова. В отличие от Snowball/Porter2, оригинальный алгоритм использует классические правила, описанные в статье M. F. Porter.

Алгоритм реализован в файле `core/porter.h` на C++ без использования STL. Ключевые функции: `measure()` --- подсчёт <<меры>> основы (чередований согласная-гласная), `is_consonant()` --- определение согласного, `has_vowel()` --- наличие гласного, `cvc()` --- проверка шаблона.

4.3. Результаты

Показатель	Значение
Уникальных токенов (до стемминга)	253 465
Уникальных основ (после стемминга)	223 983
Сокращение словаря	≈11,6%

Примеры работы стеммера:

- `mining` → `mine`
- `transactions` → `transact`
- `cryptocurrency` → `cryptocurr`
- `decentralized` → `decentr`

4.4. Выводы

Стемминг позволил сократить словарь на 11,6%, объединив различные словоформы в единую основу. Это повышает полноту поиска: запрос `mine` находит документы со словами `mining`, `mined`, `miner`.

5. Лабораторная работа №5. Закон Ципфа

5.1. Задание

Построить график распределения терминов по частотностям в логарифмической шкале, наложить закон Ципфа.

5.2. Описание метода

Закон Ципфа утверждает, что частота f слова обратно пропорциональна его рангу r: $f(r) = C/r$, где C --- константа, равная частоте самого частого слова.

Для построения графика используется скрипт `scripts/zipf.py` (Python, matplotlib). По оси X --- ранг терма, по оси Y --- его частота, обе оси в логарифмическом масштабе.

5.3. Результаты

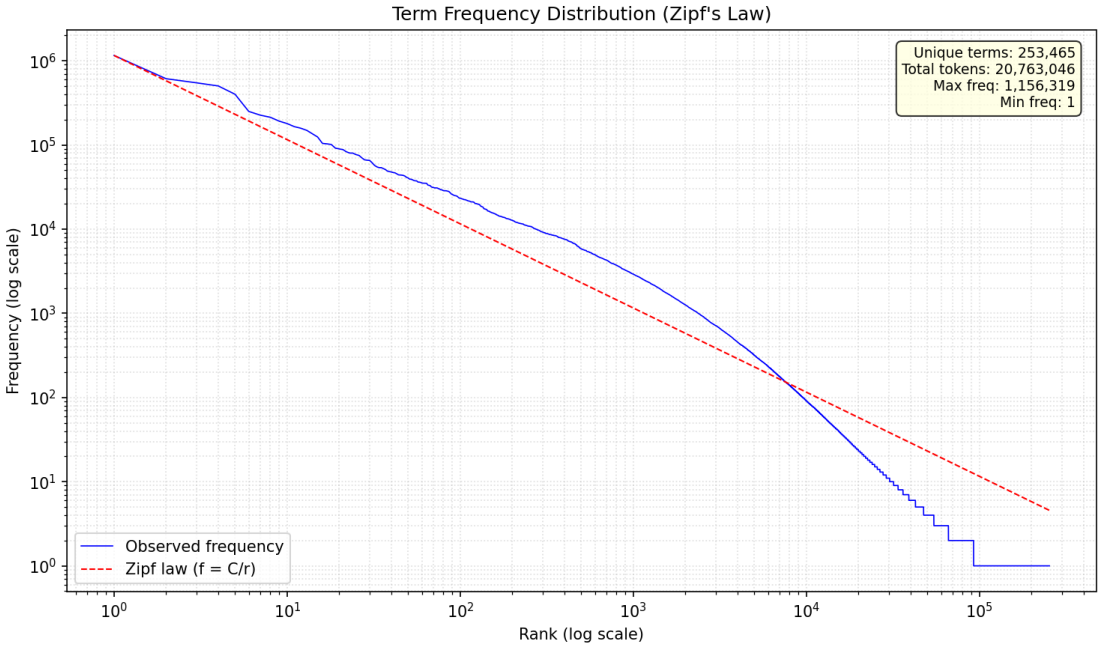

Рис. 1: Распределение терминов по частотностям (закон Ципфа), 253 465 уникальных терминов

Наблюдаемое распределение близко к теоретическому закону Ципфа. Расхождения наблюдаются:

- В области низких рангов --- частые служебные слова (`the`, `to`, `of`) чуть превышают теоретическую прямую;
- В области высоких рангов --- редкие технические термины (`defi`, `stablecoin`) отклоняются вниз из-за специфичности корпуса.

5.4. Выводы

Распределение частот терминов в корпусе криптовалютных новостей хорошо согласуется с законом Ципфа. Отклонения объясняются тематической специализацией корпуса.

6. Лабораторная работа №6. Булев индекс

6.1. Задание

Построить поисковый индекс в бинарном формате, пригодный для булева поиска.

6.2. Описание метода

Индекс реализован на C++ без STL с использованием собственных структур данных:

- **Динамический массив** (`vec.h`, структура `Vec`) --- массив с автоматическим увеличением ёмкости (начальная --- 32 элемента, рост --- $\times 2$). Сортировка --- merge sort (стабильная, $O(n \log n)$);
- **Хеш-таблица** (`map.h`, структура `Map`) --- open addressing с linear probing, хеш-функция FNV-1a (offset basis 2 166 136 261, prime 16 777 619), начальный размер --- 200 003 бакета, load factor --- 0,7;
- **Строковые утилиты** (`mystr.h`) --- `str_len`, `str_dup`, `str_cmp`, `to_lower` и др.

6.2.1. Формат индекса

Бинарный файл `index.bin` с магическим числом `YIDX`:

Секция	Смещение	Описание
Заголовок	0	Magic (4B), version (4B), num_terms (4B), num_docs (4B), fwd_offset (8B), dict_offset (8B), post_offset (8B) --- 40 байт
Прямой индекс	fwd_offset	Для каждого документа: url_len (2B), url, title_len (2B), title, source_len (2B), source
Постинги	post_offset	Для каждого терма: отсортированные doc_id (4B)
Словарь	dict_offset	Для каждого терма: term_len (1B), term, posting_offset (8B), posting_count (4B)

Формат предусматривает расширение: поле version позволяет добавлять новые секции.

6.3. Результаты

Показатель	Значение
Количество документов	33 290
Количество термов	223 983
Количество постингов	9 401 651
Размер индекса	46 МБ
Средняя длина терма	5,35 символа
Время индексации	7,28 с
Скорость	≈4 573 доков/с

6.4. Выводы

Индекс построен за 7,28 секунды. Использование хеш-таблицы с open addressing и FNV-1a обеспечивает быстрый поиск терма ($O(1)$ в среднем). Merge sort гарантирует стабильность сортировки постингов. При увеличении корпуса в 10 раз основным ограничением станет объём оперативной памяти.

7. Лабораторная работа №7. Булев поиск

7.1. Задание

Реализовать ввод поисковых запросов и их выполнение над индексом, веб-сервис и CLI.

7.2. Описание метода

Парсер булевых запросов реализован по алгоритму shunting-yard (алгоритм Дейкстры). Используются два стека --- операндов и операторов. Приоритет операций: `NOT` > `AND` > `OR`. Пробел интерпретируется как `AND`.

Поддерживаемый синтаксис:

- `&&` или пробел --- логическое И;
- `||` --- логическое ИЛИ;
- `!` --- логическое НЕ;
- Скобки для группировки.

Реализация на C++ (`core/search_cli.cpp`) без STL. Поиск терма выполняется бинарным поиском по отсортированному словарю. Операции над списками постингов --- двухуказательное слияние ($O(n + m)$).

7.3. Реализация

7.3.1. CLI

Утилита `search_cli` получает путь к индексу и выполняет запросы со стандартного ввода. Поддерживает флаги `--json` и `--limit`.

7.3.2. Веб-сервис

Веб-сервис реализован на Flask (Python). Два маршрута: `GET /` --- форма ввода, `GET /search?q=...` --- страница результатов (50 на страницу, пагинация). Для выполнения запроса вызывается `search_cli` через `subprocess`.

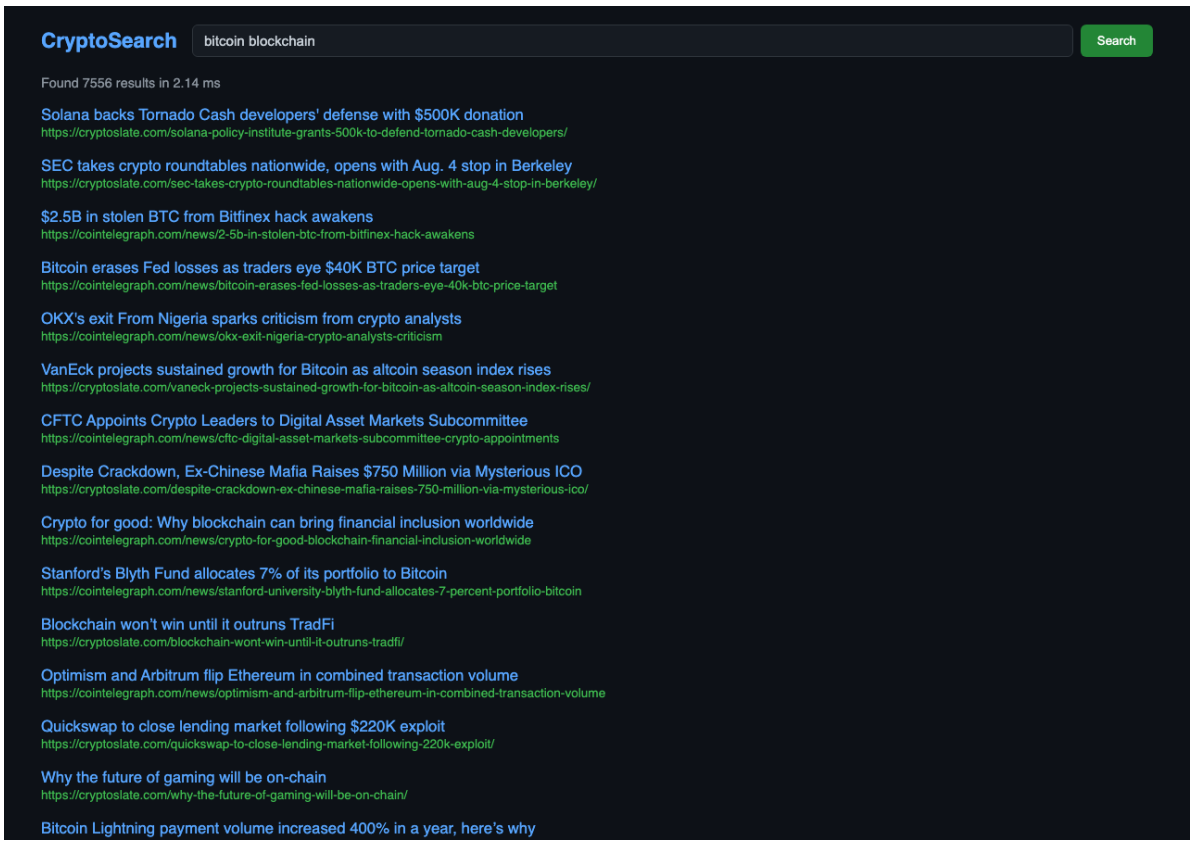

Рис. 2: Скриншот поисковой выдачи (запрос <<bitcoin blockchain>>, 7 556 результатов, 1,64 мс)

7.4. Результаты

Запрос	Результат (время)
`bitcoin`	21 711 (1,24 мс)
`ethereum | | solana`	8 196 (1,58 мс)
`bitcoin && !ethereum`	17 633 (9,96 мс)
`bitcoin blockchain`	7 556 (1,64 мс)

Запросы с оператором `NOT` выполняются дольше, так как требуют обход всего множества документов.

7.5. Выводы

Поисковая система обрабатывает запросы за 1--10 мс. Алгоритм shunting-yard корректно разбирает выражения с произвольной вложенностью скобок и переменным числом пробелов. Веб-интерфейс на Flask обеспечивает удобный доступ к результатам поиска с пагинацией.

Заключение

В рамках лабораторных работ реализована полноценная поисковая система для корпуса криптовалютных и блокчейн-новостей на английском языке:

1. Собран корпус из 33 290 документов из трёх источников (CoinTelegraph, CryptoSlate, Bitcoin Magazine);
2. Реализован асинхронный краулер с поддержкой возобновления и переобкачки;
3. Построен токенизатор со скоростью ≈38,7 МБ/с;
4. Применён стемминг по алгоритму Портера, сокративший словарь на 11,6%;
5. Подтверждено соответствие корпуса закону Ципфа;
6. Построен бинарный поисковый индекс (223 983 терма, 9,4 млн постингов, 46 МБ);
7. Реализован булев поиск с веб-интерфейсом и CLI.

Ядро поисковой системы написано на C++ без использования STL, с собственными реализациями динамического массива, хеш-таблицы (FNV-1a, open addressing), стеммера Портера и парсера запросов (shunting-yard).